

Lab 1: Introduction to R and RStudio

Applied Econometrics

Muhammad Salman Khalid, IBA Karachi

September 2026

1 Objectives

By the end of this lab you should be able to open R through RStudio, use R as a calculator, create and manipulate objects and vectors, build and inspect a data frame, import a dataset from a CSV file, install and load a package, and produce basic summary statistics and a simple plot. Everything here is a foundation for the regression work we will do in later labs, so take your time and make sure each command actually runs before moving to the next one.

2 Installing R and RStudio

R is the underlying language and engine that does the computation. RStudio is an interface that sits on top of R and makes it far more pleasant to work in. You need both, and you need to install R first.

Go to <https://cran.r-project.org> and download the version of R for your operating system, then go to <https://posit.co/download/rstudio-desktop/> and download RStudio Desktop, which is free. Install R first, then RStudio, and open RStudio rather than R itself from now on. If you already have both installed, check that RStudio recognizes a recent version of R by looking at the top of the Console pane when you open it.

3 The RStudio Layout

When you open RStudio you will normally see four panes. The Console, usually bottom left, is where commands are typed and executed one at a time and where output appears. The Source pane, top left, is where you write and save scripts so your work is reproducible rather than lost when you close R. The Environment pane, top right, lists every object currently stored in memory, which is useful for keeping track of what you have created. The Files, Plots, Packages, and Help pane, bottom right, is where plots appear and where you can browse files and installed packages.

Open a new script now through File, New File, R Script, and save it as `lab1_yourname.R`. Type every command from this lab into that script rather than directly into the console, and run each line with `Ctrl+Enter` on Windows or `Cmd+Enter` on a Mac. Working from a saved script means you can rerun your entire analysis at any point, which matters once your work gets longer than a few lines.

4 R as a Calculator

R evaluates arithmetic directly.

```
2 + 2
10 / 4
3^2
sqrt(81)
log(100)
exp(1)
```

The last three lines use built-in functions. A function in R always takes the form `function_name(argument)`, and this pattern will come up constantly, from `mean()` and `sd()` all the way to `lm()` when we get to regression.

5 Objects and Assignment

Rather than just printing a result, we usually want to store it. The assignment operator in R is `<-`, read as “gets.”

```
x <- 5
y <- 3
x + y
z <- x + y
z
```

Note that assignment does not print anything on its own. If you want to both assign and see the value, wrap the line in parentheses.

```
(z <- x + y)
```

Object names in R are case sensitive, cannot start with a number, and should be descriptive. Use `gdp_growth` rather than `x` once you are working with real variables, because a script full of single letters is unreadable a week later.

6 Data Types

R has a handful of basic data types you will run into constantly. Numeric values are ordinary numbers, either whole or decimal.

```
income <- 45000
class(income)
```

Character values are text and must be wrapped in quotes.

```
country <- "Pakistan"
class(country)
```

Logical values are `TRUE` or `FALSE`, which is what comparisons return.

```
is_employed <- TRUE
class(is_employed)
```

```
5 > 3
5 == 3
```

Note the double equals sign for testing equality. A single = is used for assignment in some contexts and will not test equality.

7 Vectors

A vector is an ordered collection of values of the same type, and it is the building block of almost everything in R, including every column of a dataset. Vectors are created with `c()`, for “combine.”

```
ages <- c(22, 25, 19, 31, 28)
ages
```

Individual elements are accessed with square brackets, and R indexing starts at 1, not 0.

```
ages[1]
ages[3]
ages[2:4]
```

A set of functions summarizes a numeric vector directly.

```
length(ages)
sum(ages)
mean(ages)
sd(ages)
min(ages)
max(ages)
sort(ages)
```

Vectors can also hold character data.

```
cities <- c("Karachi", "Lahore", "Islamabad", "Multan")
cities
cities[2]
```

Arithmetic on a numeric vector is applied element by element, which is one of the more useful features of the language.

```
ages + 1
ages * 2
ages > 25
```

That last line returns a logical vector, and logical vectors are how we will filter data in a few minutes.

8 Data Frames

A data frame is a table, essentially a spreadsheet held in memory, where each column is a vector and all columns have the same length. Almost every dataset you touch in this course will be a data frame.

```
name <- c("Ali", "Sara", "Bilal", "Ayesha", "Hassan")
age <- c(22, 25, 19, 31, 28)
city <- c("Karachi", "Lahore", "Karachi", "Islamabad", "Lahore")
income <- c(35000, 62000, 21000, 88000, 45000)

students <- data.frame(name, age, city, income)
students
```

A few functions let you inspect a data frame without printing the whole thing, which matters once you are working with thousands of rows.

```
str(students)
head(students, 3)
summary(students)
dim(students)
names(students)
```

Individual columns are accessed with the dollar sign.

```
students$age
mean(students$income)
```

Rows can be filtered using a logical condition, which is one of the most common operations you will perform.

```
students[students$city == "Karachi", ]
students[students$income > 40000, ]
```

9 Installing and Loading Packages

Base R does a great deal, but most applied work relies on packages that extend it. A package is installed once per machine with `install.packages()` and then loaded in every new session with `library()`.

```
install.packages("dplyr")
library(dplyr)
```

You only need to run `install.packages()` again if you reinstall R or move to a new computer, but `library()` has to be run at the start of every session, since R does not keep packages loaded between sessions. We will use `dplyr` and `ggplot2` extensively later in the course, so it is worth installing both now.

```
install.packages("ggplot2")
library(ggplot2)
```

10 Importing Data from a CSV File

In practice you will almost never type data in by hand, you will read it in from a file. The most common format is CSV. First set your working directory to the folder containing your file, either through the Session menu in RStudio or with `setwd()`, then read the file in.

```
getwd()
setwd("path/to/your/folder")
mydata <- read.csv("dataset.csv")
head(mydata)
str(mydata)
```

If R cannot find the file, the most common cause is that the working directory does not match the folder where the file actually sits, so check `getwd()` first and confirm the file name and extension match exactly, including capitalization.

11 A First Plot

A histogram is a quick way to see the distribution of a variable.

```
hist(students$income,
     main = "Distribution of Student Income",
     xlab = "Income (PKR)",
     col = "steelblue")
```

A scatterplot shows the relationship between two variables, which is where we are ultimately headed in this course.

```
plot(students$age, students$income,
     main = "Age vs Income",
     xlab = "Age",
     ylab = "Income (PKR)",
     pch = 19)
```

12 Getting Help

If you are unsure what a function does, put a question mark in front of its name and run it, which opens the help file in the Plots and Help pane.

```
?mean
?read.csv
```

When you get an error, read it carefully before searching for it. R error messages are usually specific about what went wrong, and most of the errors you hit in this course will be a missing package, a misspelled object name, or a mismatched working directory.

13 In-Lab Exercises

Complete these during the lab session using the `students` data frame created above.

1. Create a vector called `gpa` with five values of your choosing between 2.0 and 4.0, and compute its mean and standard deviation.
2. Add `gpa` as a new column to the `students` data frame.
3. Find the average income of students in Lahore only.

4. Create a new logical column called `high_income` that is `TRUE` when income exceeds 50000 and `FALSE` otherwise.
5. Produce a histogram of the `age` column with a title of your choosing.

14 Take-Home Exercise

Find any dataset online in CSV format, for instance from the Pakistan Bureau of Statistics, the World Bank Open Data portal, or the State Bank of Pakistan, and save it to a folder on your computer. Write a short script that sets your working directory, imports the file with `read.csv()`, reports its dimensions with `dim()`, prints the structure with `str()`, computes the mean and standard deviation of one numeric variable of interest, and produces one histogram of that variable. Save your script and bring it to the next lab.

15 Looking Ahead

The next lab moves from describing a single variable to describing the relationship between two, and introduces `lm()`, the function we will use for ordinary least squares regression for the rest of the course. Everything covered today, vectors, data frames, and filtering, is what `lm()` operates on underneath, so make sure the material above feels comfortable before the next session.